

Assignment #4

join

2019060546

주원웅

• Design

Merge 조인을 사용하는 것이 가장 빠르게 구현할 수 있는 방법이라고 생각했다.

B+ Tree 의 경우 인덱스가 모두 Sort 되어있기 때문에 머지 조인을 사용할 수 있고, 머지조인의 경우 이미 정렬되어 있는 경우 퍼포먼스가 뛰어나다고 생각했기 때문이다.

루트페이지, 헤더페이지, 파일디스크립터를 모두 하나씩 더 선언하여 한 테이블이 아니라 두 테이블을 관리할 수 있도록 하였다.

Page를 Load 하는 모든 함수에 parameter 를 추가하여, 두 테이블을 open 할 수 있도록 구현하였다.

Insert, delete, find 함수의 경우 사용하지 않기때문에 main 함수와 bpt 소스코드에서 모두 주석처리하였다.

• Implement

1. Join 함수

```
void db_join()
{
    // clock_t start, end;
    // start = clock();
    // double cpu_time_used;
    page *p1;
    page *p2;
    off_t loc1 = hp1->rpo;
    off_t loc2 = hp2->rpo;

    // printf("left = %ld, key = %ld, right = %ld, \n",
    // rt->b_f[0].key, rt->b_f[0].p_offset, rt->is
    if (rt1 == NULL)
    {
        // printf("Empty tree.\n");
        return;
    }

    p1 = load_page(loc1, 1);
    p2 = load_page(loc2, 2);
    // printf("rt->is_leaf: %d\nrt->num_of: %d\n", r
    while (!p1->is_leaf)
    {
        loc1 = p1->next_offset;
        free(p1);
        p1 = load_page(loc1, 1);
    }
    while (!p2->is_leaf)
    {
        loc2 = p2->next_offset;
        free(p2);
        p2 = load_page(loc2, 2);
    }
    int i = 0;
    int j = 0;
```

p1, p2 를 각각 하나씩 두어서
table1, table2 에 대한 접근을 했다.

반복문을 통해서 가장 왼쪽에 존재
하는(가장 인덱스가 낮은) leaf를 찾
았다.

key 값이 이미 정렬되어 있기에
가장 작은 것부터 읽어가며
next_offset 을 확인하면 모든 index
를 살펴볼수 있을 것이라고 생각했
다.

변수 i, j 는 각각 table1, table2 의
leaf node의 키를 탐색하기 위한 변
수이다.

```
// merge join
while (!(p1->next_offset == 0 && i == p1->num_of_keys) && !(p2->next_offset == 0 && j == p2->num_of_keys))
{
    if (p1->records[i].key == p2->records[j].key)
    {
        printf("%d,%s,%s\n", p1->records[i].key, p1->records[i].value, p2->records[j].value);
        j++;
    }
    else if (p1->records[i].key < p2->records[j].key)
    {
        i++;
    }
    else if (p1->records[i].key > p2->records[j].key)
    {
        j++;
    }
    if (i == p1->num_of_keys)
    {
        loc1 = p1->next_offset;
        free(p1);
        p1 = load_page(loc1, 1);
        i = 0;
    }
    if (j == p2->num_of_keys)
    {
        loc2 = p2->next_offset;
        free(p2);
        p2 = load_page(loc2, 2);
        j = 0;
    }
}
free(p1);
free(p2);
```

이후 p1, p2 둘 중 하나라도 끝까지 도달하면 중지하도록 구현했다.

둘의 키 값이 동일한 경우 출력하고 table2의 인덱스를 하나 증가하게 하였고, table1의 key 값이 더 작다면 i++, table2의 key 값이 더 작다면 j++ 하여 같은 경우만 출력할 수 있도록 구현하였다.

만약 현재 노드의 num_of_key 만큼 탐색이 완료되면 다음 next_offset의 오른쪽 노드로 이동하도록 구현하였다.

```
int main()
{
    int64_t input;
    char instruction;
    char buf[120];
    char *result;

    // Assignment 4
    open_table("table1.db", "table2.db");
    // "table1.db", "table2.db"라는 두 파일을 열 수 있게 수정

    while (scanf("%c", &instruction) != EOF)
    {
        switch (instruction)
        {
            case 'i':
                scanf("%ld %s", &input, buf);
                // db_insert(input, buf);
                break;
            case 'f':
                scanf("%ld", &input);
                // result = db_find(input);
                if (result)
                {
                    printf("Key: %ld, Value: %s\n", input, result);
                }
                else
                {
                    printf("Not Exists\n");
                }
                fflush(stdout);
                break;
            case 'd':
                scanf("%ld", &input);
                // db_delete(input);
                break;

            // Assignment 4
            case 'j':
                db_join();
                break;

            //////////////////////////////////////
        }
    }
}
```

Main 함수에서 insert, find, delete 모두 주석처리하였고, join만 사용하도록 하였다.

• Result

```
199746,1,22
199752,1,22
199758,1,22
199764,1,22
199770,1,22
199776,1,22
199782,1,22
199788,1,22
199794,1,22
199800,1,22
199806,1,22
199812,1,22
199818,1,22
199824,1,22
199830,1,22
199836,1,22
199842,1,22
199848,1,22
199854,1,22
199860,1,22
199866,1,22
199872,1,22
199878,1,22
199884,1,22
199890,1,22
199896,1,22
199902,1,22
199908,1,22
199914,1,22
199920,1,22
199926,1,22
199932,1,22
199938,1,22
199944,1,22
199950,1,22
199956,1,22
199962,1,22
199968,1,22
199974,1,22
199980,1,22
199986,1,22
199992,1,22
199998,1,22
q
```

Table1 에 2의 배수 key를 100000개, Table2
에 3의 배수 key를 70000개 insert 하여 두 테
이블을 join 한 결과이다.

결과는 6의 배수 key만 나오도록 되었고, 각각
table1, table2 의 value 값을 출력하였다.

• Trouble shooting

1. 페이지를 Load 하는 과정에서 조금 많이 꼬였었다.
2. Load 함수를 수정하니 insert, delete, find 함수에서 컴파일 에러가 많이 발생하여 수정할 부분이 너무 많아 주석처리했다.